NESNE YÖNELİMLİ PROGRAMLAMA

Ders 5: Nesne Yönelimli Programlama I

Property (Özellik) Kavramı

- Hatırlatma (Paketleme - Encapsulation)
  - Private bir değişkene bir metot yazarak üzerinden ulaşabilir veya başka bir metot yazarak değişkeni değiştirebiliriz.
  - Bu işlemi yapmamızın nedeni değişkeni kontrol altına almaktı. Bu sayede y istenmeyen durumların oluşması engellenmiş olur. (Aşağıdaki örnek için <0 girilmesini önleme)

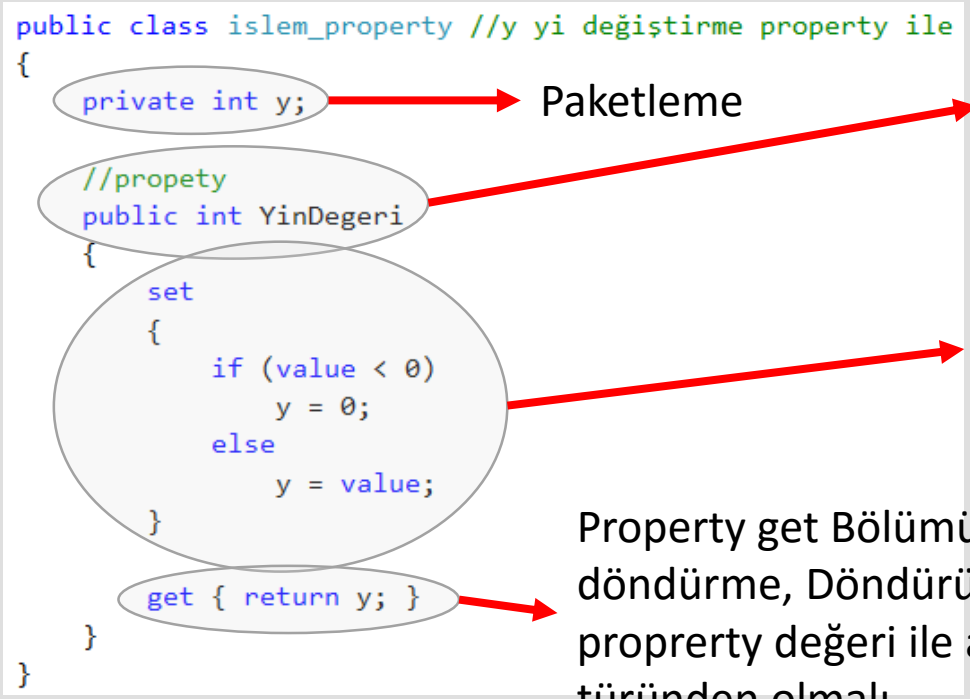
```
public class islem_fonksiyon //y yi değiştirme fonksiyon ile
{
    private int y;

    public void y_degistir(int m)
    {
        if (m < 0) //sadece pozitif y alır
            y = 0;
        else
            y = m;
    }

    public int y_goster()
    {
        //return y;
        return this.y; //this bulunulan sınıfı işaret eder.
    }
}
```

Property (Özellik) Kavramı

- Bir önceki işlem standart hale gelmiş ve tek yordam ile iki yordamı temsil nasıl ederiz diye düşünülmüş ve Property kavramı önerilmiş.
- Property bir sınıfın içindeki private veya public (tavsiye edilmez) alanları yöneten, yönlendiren ve onlara ait özellikleri belirleyen tek yordamdır. Aynı yordamda hem değiştirme hem de belirli bir değişkeni döndürme işlemi yapılabilir.
- Bu işlemler Property yordam içinde set ve get bölümlerine yazılır.



Paketleme

Property: bir yordam gibi parantez kullanılmaz

Property set Bölümü:Main bölümünde property'e bir değişken gibi ulaşılır.
value eşittir'den sonra yazılan değerdir.

Property get Bölümü: Değeri döndürme, Döndürülen değer proprerty değeri ile aynı veri türünden olmalı.

İki yordamlı durumdan tek yordamlı Property'e geçiş

- Her iki sınıf aynı işlemleri yapmaktadır. Sadece çağrılmaları farklıdır. Property sınıfınıza rahat kullanım katmak için kullanılabilir.

```
public class islem_fonksiyon //y yi değiştirme fonksiyon ile
{
    private int y;

    public void y_degistir(int m)
    {
        if (m < 0) //sadece pozitif y alır
            y = 0;
        else
            y = m;
    }

    public int y_goster()
    {
        //return y;
        return this.y; //this bulunulan sınıfı işaret eder.
    }
}
```

```
public class islem_property //y yi değiştirme property ile
{
    private int y;

    //propety
    public int YinDegeri
    {
        set
        {
            if (value < 0)
                y = 0;
            else
                y = value;
        }
        get { return y; }
    }
}
```

```
islem_fonksiyon _is = new islem_fonksiyon();
_is.y_degistir(90);
Console.WriteLine(_is.y_goster());
```

Metot üzerinden değiştirme ve değeri döndürme

```
islem_property _ip = new islem_property();
_ip.YinDegeri = 90;
Console.WriteLine(_ip.YinDegeri);
```

= ile set alanı çalışır.
Değer döndürme için get alanı çalışır.
Metottan farkı parantez kullanılmaması!!!

Property Kavramı

- Bir property her zaman get veya set alanlarını içermek zorunda değildir. Bu tamamen belirlediğiniz property durumuna göre değişebilir.
- İçinde sadece get alanı bulunduran bir property read-only (sadece okunabilir)'dir.
- İçinde sadece set alanı bulunduran bir property write-only (sadece yazılabilir)'dir.
- Özellikle bazı durumlarda sadece okunabilir property'ler çok fazla kullanılmaktadır.

Statik Kavramı

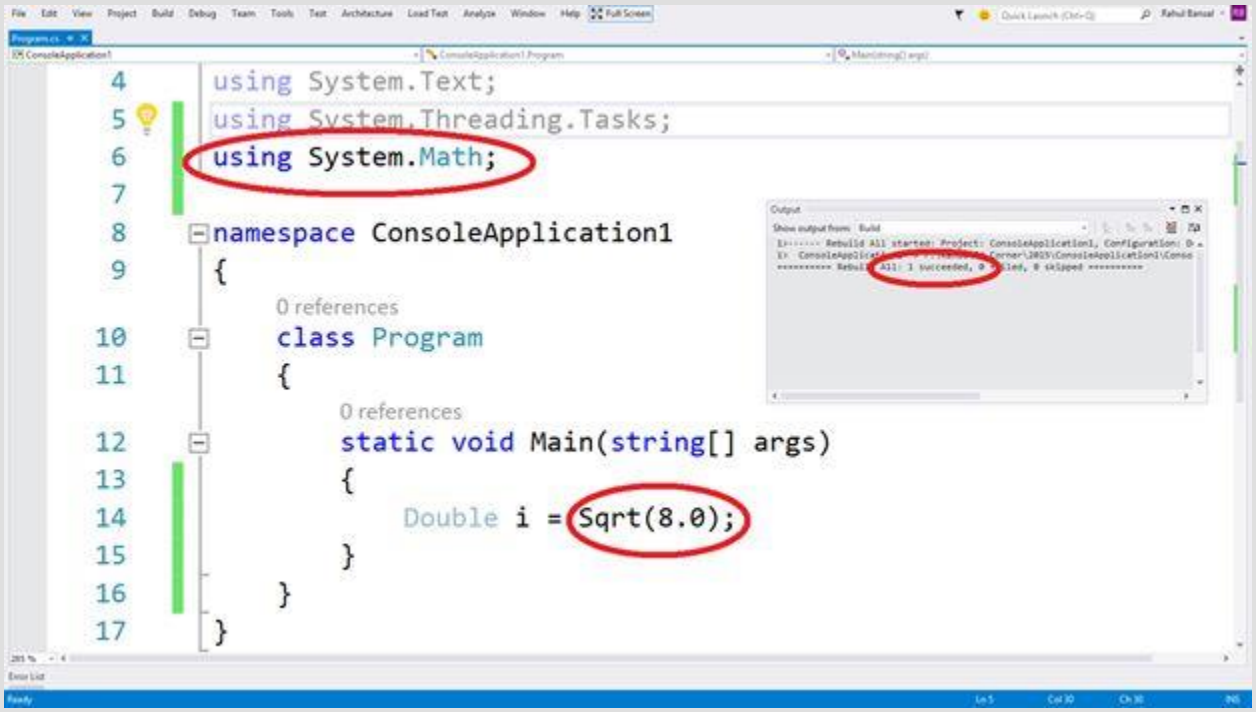
- İçinde bulunduğu sınıftan nesne oluşturulmadan veya hiç bir nesneye referans olmadan kullanılabilen üyeler static olarak nitelendirilir.
- Static olarak tanımlanan bir metoda program çalıştığı sürece erişilir, böylece sadece bir metot ile birden çok nesne çağırılır.

Static Kavramı

- Bir sınıf(class) içerisinde Metotlar, değişkenler ve sınıflar static olarak tanımlanabilir.
- Bir sınıfa ait static bir yapıcı metod (constructors) tanımlanabilir.
- Değişmezler (constants) bilinçsiz olarak (implicit) static tanımlanmışlardır.
- Readonly referanslar açıkça belirtilmedikçe static değildirler.

Hazır Static Metodlar

- C# içinde new ile yaratılmadan direkt kullanılan üyeler statictir.
- Örneğin Math kütüphanesi içindeki Sqrt metotu direkt kullanılır.



Static - Main

- static olma durumunun en bilinen örneği Main() metodudur. Main(), herhangi bir nesne oluşturulmadan önce çağrılması gerektiği için static olarak tanımlanmıştır. Başka bir deyişle de bir nesne metodun üreteceği sonucu etkilemeyecek ise o metot static olarak tanımlanır.

```
class Program
{
    static void Main(string[] args)
    {

    }
}
```

- Program çalıştırıldığında çalışan ana kısımdır.
- Program kapatılmadan sonlanmaz.

Static ve Harici Değişken

- nesne oluşturulmadan veya hiç bir nesneye referans olmadan kullanılabilen üyeler static olarak nitelendirilir.

```
class Program
{
    //Harici veri
    public int x = 90;

    //Static veri
    public static int y = 90;

    static void Main(string[] args)
    {
        Program _p = new Program();

        Console.Write(_p.x); //Nesne oluşturup çağırdık. Bellek daha etkin kullanılır.

        Console.Write(y); //Nesne oluşturmadan çağırdık. Daha hızlıdır.

        System.Console.ReadLine();
    }
}
```

Kendi içinde bulunduğu sınıftan nesne yaratma
Harici sınıfa başka ulaşabilme şansınız yok

Harici sınıfın içinde static değişken veya metot olabilir.
Main metodu static'tir ve harici sınıf içindedir.

Static Sınıf ve Harici Sınıf

- Static sınıfın tüm üyeleri (metot ve değişkenleri) static olmak zorundadır.
- Harici sınıfın içinde static değişken veya metot olabilir. Harici bir sınıf içindeki static bir metot veya değişkeni çağırma işleminde nesne yaratılmadan çağrılır.

```
public static class Matematik_Static
{
    public static int topla(int x, int y)
    {
        return x + y;
    }
}
```

Static bir sınıf, tüm üyeleri static.

```
public class Matematik_Harici
{
    public int topla(int x, int y)
    {
        return x + y;
    }

    public static int topla2(int x, int y)
    {
        return x + y;
    }
}
```

Harici bir sınıf ve bir üyesi harici, diğer üyesi static.

Static ve Harici Sınıf

```
public static class Matematik_Static
{
    public static int topla(int x, int y)
    {
        return x + y;
    }
}
```

```
public class Matematik_Harici
{
    public int topla(int x, int y)
    {
        return x + y;
    }

    public static int topla2(int x, int y)
    {
        return x + y;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine(Matematik_Static.topla(5,6)); //Static sınıftan static fonksiyon çağırma.
        //statik sınıfa statik olmayan fonksiyon tanımlanamaz.

        Matematik_Harici _m = new Matematik_Harici();

        Console.WriteLine(_m.topla(15, 16)); //harici sınıftan harici bir metodu çağırma

        Console.WriteLine(Matematik_Harici.topla2(35, 36)); //harici sınıftan static bir metodu çağırma

        Console.ReadLine();
    }
}
```

Static Constructor

```
public static class Matematik_Static
{
    public static double Pi = 3.14;

    static Matematik_Static()
    {
        Pi = Math.PI;
    }
}

public class Matematik_Harici
{
    public static double Pi = 3.14;
    static Matematik_Harici()
    {
        Pi = Math.PI;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine(Matematik_Static.Pi); //Sonuç her iki yöntemde aynıdır.
        //Constructor newle değil program ilk çalışırken devreye girer. Main gibi
        Console.WriteLine(Matematik_Harici.Pi);

        Console.ReadLine();
    }
}
```

Static constructor erişim belirteci yoktur.

Static - Const ve Readonly

```
class Program
{
    //Harici veri
    public int x = 90;

    //Static veri
    public static int y = 90;

    public const double E = 2.7;
    public readonly double F = 1.1;
    public readonly static double G = 1.8;

    static void Main(string[] args)
    {
        Program _p = new Program();

        Console.WriteLine(_p.x); //Nesne oluşturup çağırdık. Bellek daha etkin kullanılır.
        Console.WriteLine(Program.y); //Nesne oluşturmadan çağırdık. Daha hızlıdır.
        Console.WriteLine(Program.E); //Const ifade direkt statictir.
        Console.WriteLine(_p.F); //readonly bir ifade static değilse nesne üzerinden çağrılır.
        Console.WriteLine(Program.G); //readonly bir ifade static ise direkt çağrılır.

        System.Console.ReadLine();
    }
}
```

Niye Static

- Main eğer static olmasaydı, main'i kim yaratacaktı! Bir NYP uygulamasında en az bir üye static olmalıdır ve bu üye ilk çalışan bölüm olmalıdır.
- Çok sık kullanılan harici sınıflar ihtiyaç durumunda belleğe yüklenir. (new) Eğer bu sınıfı veya sınıf içindeki üyeyi çok sık kullanıyorsak static olarak tanımlayabiliriz. Ama static üyelerin yazılım çalıştığı sürece bellekte yer kapladığını unutmayın. Harici sınıflar ise GC tarafından kullanılmıyorsa yok edilirler.
- Eğer bir sınıftan çok fazla sayıda nesneye ihtiyaç varsa static tanımlanmaz.

NameSpace (İsim Uzayı)

- Programlama dillerinde bir takım hazır kütüphaneler mevcuttur. Bu kütüphanelerin bazıları standart olmasının yanı sıra bazıları da programcılar tarafından oluşturulmuştur.
- Bu konuda Microsoft'un geliştirdiği .NET Framework adlı bir alt yapı kullanılmaktadır. Bu alt yapı Microsoft'un geliştirdiği programlama dillerini kullanan programcılara temel türler ve sınıflar sunmuştur.
- .Net bütün bu sınıfları ve türleri iyi bir şekilde organize edebilmek için isim uzayı (namespace) kavramını sıklıkla kullanır.

```
using System;

namespace Ders05_01_Static
{
    class Program
    {
        static void Main(string[] args)
        {

        }
    }
}
```

İsim uzayı çağırma

Sınıfımızın isim uzayı

NameSpace

- İsim uzayları (Namespaces) C# dilinde using anahtar sözcüğü ile kullanılır.

  using İsim_Uzay_İsmi;

- Bir yazılımcı yazdığı kodların organizasyonunda isim uzaylarını (namespaces) kullanabilir. Tür ismi ya da sınıf sadece kendi isim uzayları (namespaces) içinde görünürlüğe sahip olduğundan isim uzaylarının (namespaces) kullanılması tür isimlerinin ya da sınıf isimlerinin karışmasını engeller. Yazılan sınıflar için de bir isim uzayı (namespace) tanımlanarak kaynak kod istenilen şekilde organize edilebilir.

Nested NameSpaces

- Ayrıca .Net Framework ile içiçe (nested) isim uzayları (namespaces) tanımlanmıştır. Böylece yazılımcı hazırladığı kodları çok daha rahat organize edebilmektedir. İçiçe (nested) isim uzayları (namespaces) ile hiyerarşik bir düzene sahip kod blokları yaratılır; bu da yazılımcının işini oldukça kolaylaştırmaktadır.
- çiçe (nested) isim uzayları (namespaces) programa eklenirken de "using" anahtar sözcüğü kullanılır. Tek fark içteki bir isim uzayı (namespace) kullanılmak istendiğinde "using" anahtar sözcüğüyle birlikte isim uzayının (namespace) tam yolu yazılmalıdır. Şu şekilde örneklendirilebilir:

  using NameSpace_Ust.NameSpace_Alt_01;

- NameSpace ismi noktalı bir biçimde verildiğindede alt namespace anlamına gelir. Örneğin:

namespace NameSpace_Ust.NameSpace_Alt_03;

```
namespace NameSpace_Ust
{
    namespace NameSpace_Alt_01
    {
        class Sinif01
        {
            //kod
        }

        class Sinif02
        {
            //kod
        }
    }

    namespace NameSapce_Alt_02
    {
        class Sinif03
        {
            //kod
        }
    }
}
```

Soru

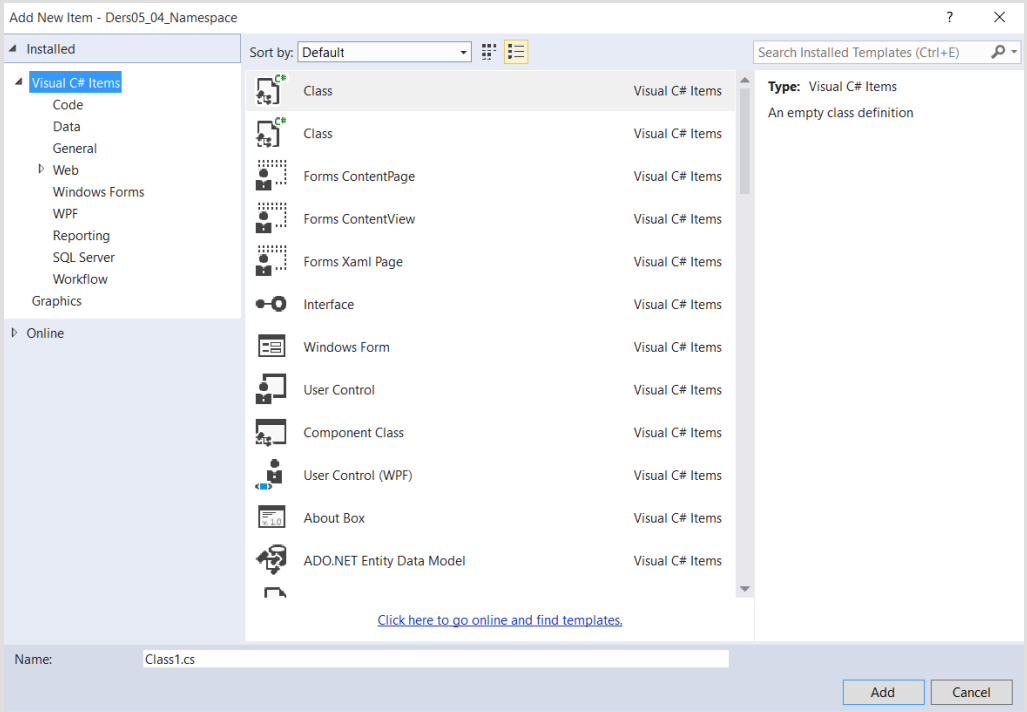
- Console diye bir değişken tanımlandınız.
- Console.WriteLine çağrılabilir mi?

Cevap

- Console adında bir değişken tanımlandığında System.Console içindeki WriteLine çağrılamaz. Local değişken Console System.Console'un içindeki yerini almıştır. Using anlamını kaybetmiştir.
- Ancak, WriteLine çağırmak için System.Console.WriteLine türünden bir çağırma işlemi yapılabilir. Tam yol belirtildiği için Console'un hangi konsol olduğunu sistem anlayacaktır.

Farklı bir cs dosyasında sınıf yaratma

- Proje üzerinde sağ tuş Add -> Add New Item deyip projenize yeni bir sınıf dosyası ekleyebilirsiniz.
- Aynı Proje içinde NameSpace'leri aynı ise direkt ulaşabilirsiniz. Eğer NameSpace'leri farklı ise ulaşmak istediğiniz alandan NameSpace ismi ile veya using bölümüne NameSpace yazarak ulaşabilirsiniz.



Aynı Proje içindeki NameSpace'i kullanma

```
using System;

using Matematik;

namespace Ders05_04_Namespace
{
    class Program
    {
        static void Main(string[] args)
        {
            Matematik.temel_islemler _ti = new Matematik.temel_islemler(); //Using olmadan çağırma

            Console.WriteLine(_ti.topla(5,6));

            diger_islemler _di = new diger_islemler(); //using matematik üzerinden çağırma

            Console.WriteLine(_di.kare_al(5));

            Console.ReadLine();
        }
    }
}
```

Program.cs

```
namespace Matematik
{
    public class temel_islemler
    {
        public int topla(int x, int y)
        {
            return x + y;
        }

        public int cikar(int x, int y)
        {
            return x - y;
        }
    }

    public class diger_islemler
    {
        public int kare_al(int x)
        {
            return x * x;
        }

        public int kup_al(int x)
        {
            return x * x * x;
        }
    }
}
```

Matematik.cs

Bir namespace içinde birden fazla namespace veya sınıf olabilir. Yukarıdaki örnekte aynı proje içinde iki ayrı dosya gözüküyor. Direkt using bölümüne diğer dosyadaki namespace yazıyoruz. Eğer yazmasak bile NameSpace ismi ve sınıf ismi şeklinde using kullanmadan çağırabiliriz.

Eğer Matematik Dosyasındaki NameSpace'i Ders_05_04_Namespace yaparsak using kullanmadan direkt sınıflara ulaşabiliriz.

Farklı Projelerin Sınıflarını Kullanma

DLL kullanımı

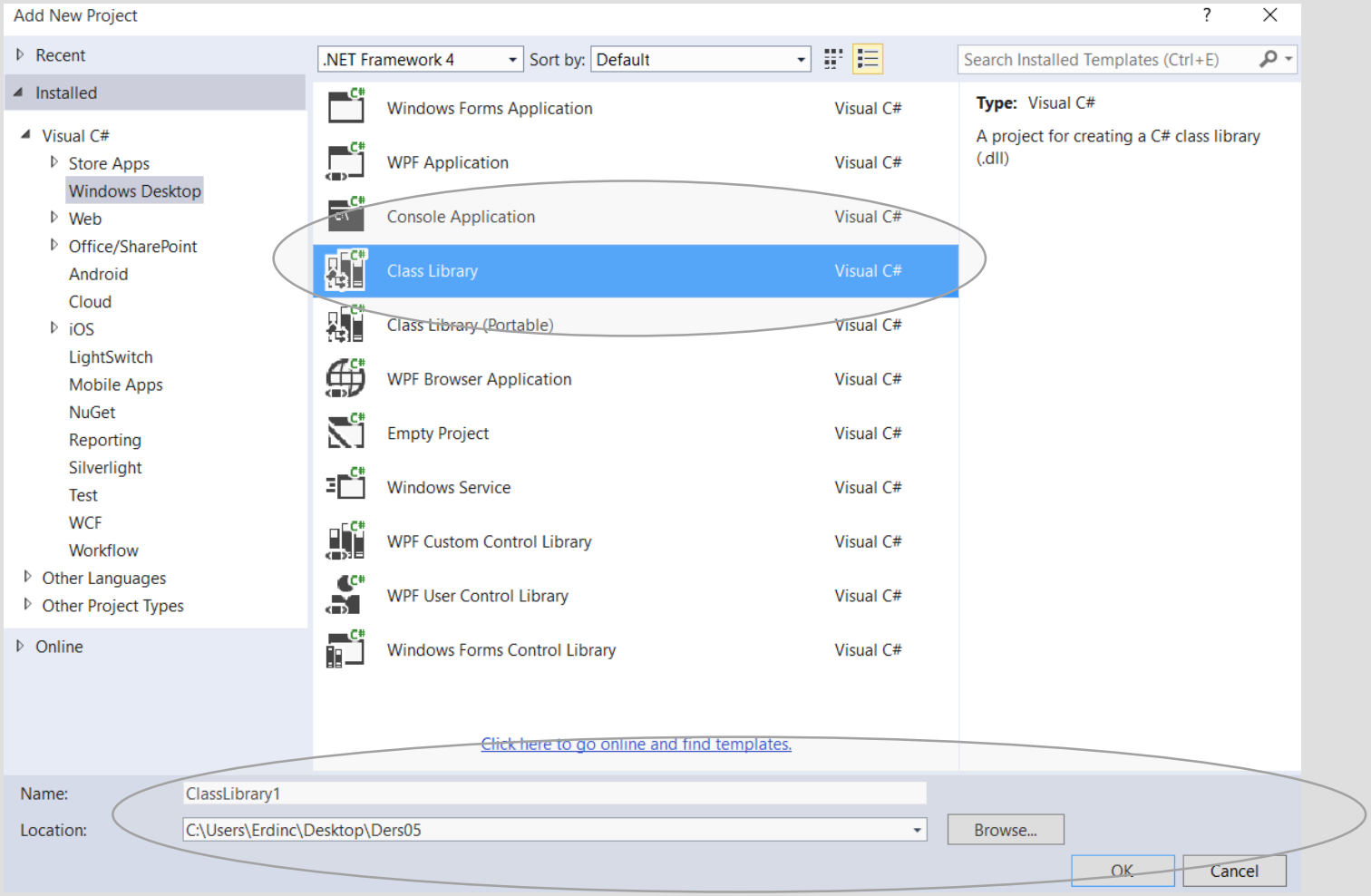
- Farklı bir projenin sınıfı kullanmak için DLL (Dynamic Link Library) dosya kavramını bilmemiz gereklidir.
- DLL
  - çalışabilen kod bulunduran
  - kendi başına çalışmayan
  - çalışabilir (exe-executable) dosyalar tarafından dinamik veya statik olarak çağrılabilen
  - içinde namespace, sınıf, değişken ve metotlar barındıran dosyalardır.
- İşletim sistemi içinde, program dosyaları içinde bir çok DLL dosya görebilirsiniz. Bu bölümden kendi DLL oluşturacağız.

Niçin DLL

- DLL iyi şekilde hazırlandığında bir çok farklı program tarafından kullanılabilir. (Reusability)
- Çalışabilir dosyalarının boyutunun şişmesini engeller.
- Farklı ortamlara rahat şekilde taşınabilir.
- Farklı yazılımların DLL, DLL tanımak kaydıyla kullanılabilir. Bu sayede bazı işlemler çok hızlı şekilde yapılabilir.
- Takım çalışmasını destekler. Takım arkadaşlarınızın yazdığı DLL kullanabilirsiniz.

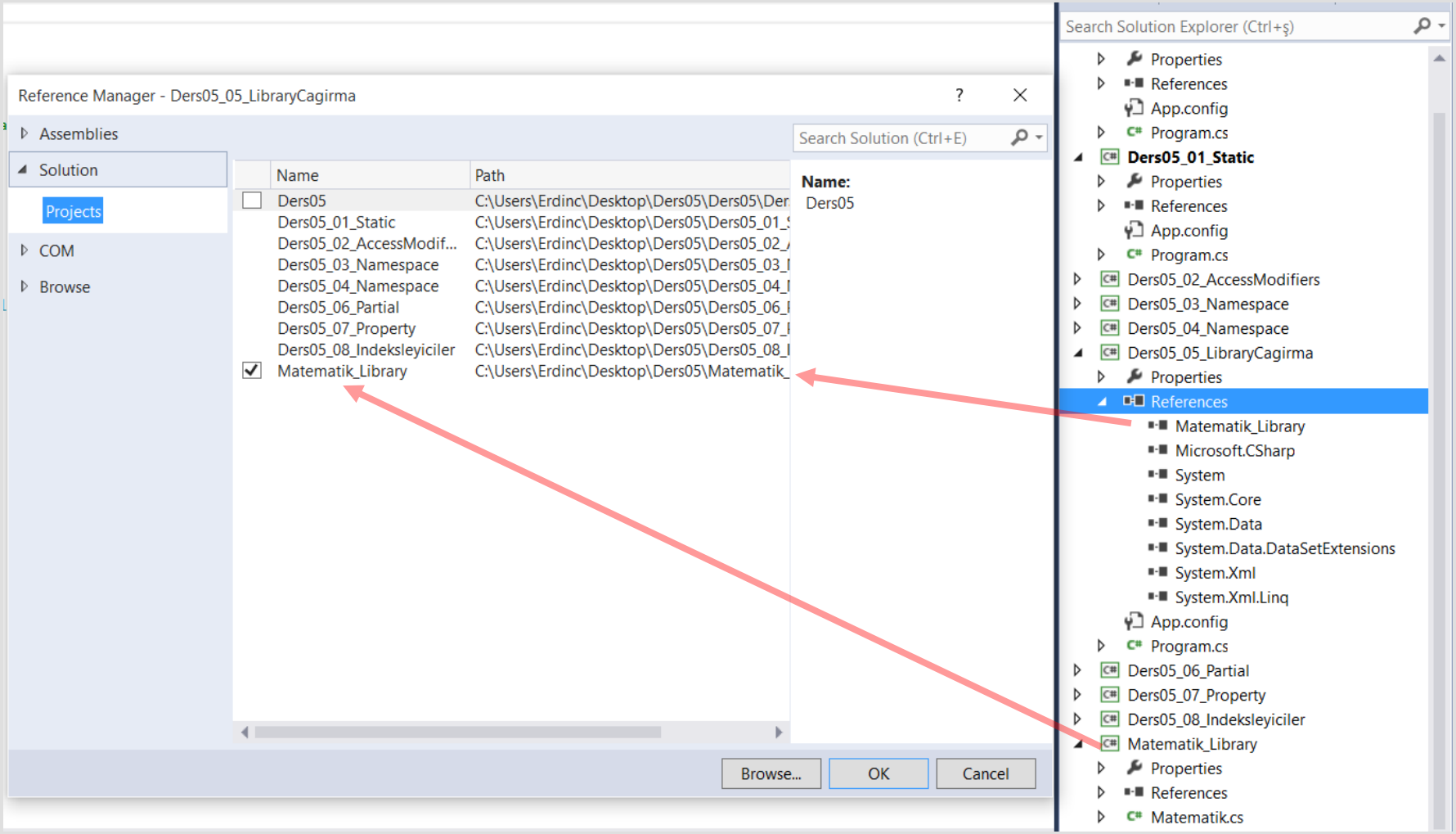
DLL Oluşturma

- Öncelikle Solution Explorer yeni bir proje açalım. Bu projede «Class Library» seçip DLL bir isim verelim. DLL içine istediğimiz kadar namespace, sınıf, değişken ve yordam yazabiliriz.



DLL diğer projeden çağırma

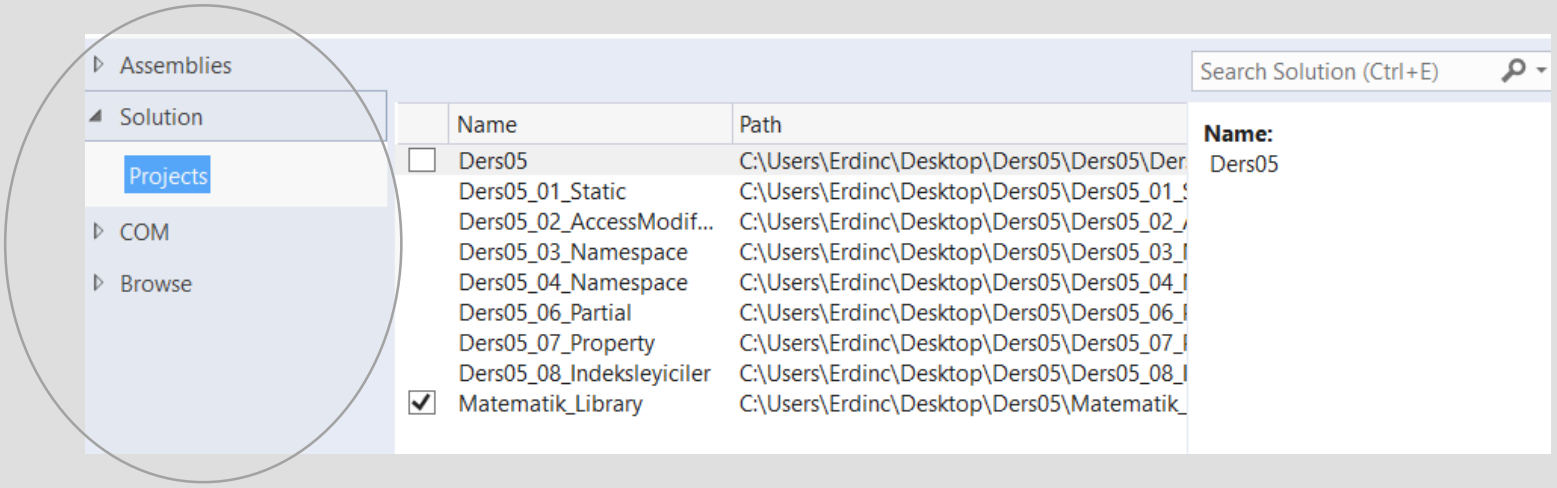
- DLL çağırmak istediğimiz projede References bölümünde Add Reference deyip ekleme yaparız.



DLL Kullanma

- Solution Projects: Projenizde bulunan kendi DLL'lerinize
- Assemblies: .Net Framework bileşenlerine (component)
- COM: COM bileşenlerine
- Browse: Bilgisayarınızda bulunan DLL'lere veya arkadaşınızdan aldığınız .Net DLL yükleyebilirsiniz

ulaşabilirsiniz.



DLL Kullanma

- Using bölümüne DLL namespace/leri eklenebilir.
- Bir veya birden fazla sınıf çağrılabilir.
- Eğer Namespace using bölümüne tanımlı değilse NameSpace İsmi ile direkt kullanılabilir.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Matematik_Library; //Reference bölümünden add yapıp eklenince çağırabiliriz.

namespace Ders05_05_LibraryCagirma
{
    class Program
    {
        static void Main(string[] args)
        {
            diger_islemler _di = new diger_islemler(); //using'te tanımlandğı için direkt çağrılabilir.

            Console.WriteLine(_di.kup_al(6));

            Matematik_Library.
        }
    }
}
```

DLL Kullanımı

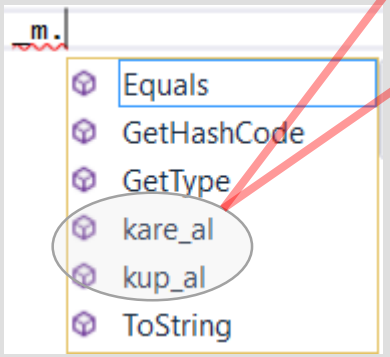
- C# veya .Net ortamında bir DLL kullanılabilmesi için .Net ortamında yazılmış olması gerekir.
- VB.Net, F#.Net, Cobol.Net vs. gibi programlama dillerinde yazılan DLL'ler tüm .Net dilleri ile kullanılabilir.
- Arkadaşınızın veya internet bulduğunuz .Net ortamında bulduğunuz bir DLL'i References bölümüne Add Reference yaptıktan sonra kullanıabilirsiniz.

Partial Class (Parçalı Sınıf)

- Büyük projelerde sınıfların karışık yapıda olmasını engellemek için sınıfı belirli parçalara bölmek daha mantıklı bir yol olacaktır.
- Ayrıca, takım çalışması açısından aynı sınıfın bir kısmını biri diğer kısımları başka biri veya birileri yazabilir.
- Sınıflarımızı C# taki "partial" anahtar kelimesi ile parçalara bölebiliriz. Parçaya ayırmayı düşündüğümüz sınıfları partial kelimesini class başına eklememiz yeterlidir.
- Partial class farklı dosyalarda olabilir. Ancak aynı namespace içinde olmalıdır.

Partial Class

- Değişkenler ve metotlar ayrı iki sınıfta gibi görünse de main bölümünde bir nesne oluşturduğumuzda tek sınıf içinden metotları çağırır gibi oluruz.




```
namespace Ders05_06_Partial
{
    //Partial 1
    public partial class Matematik
    {
        public int kare_al(int x)
        {
            return x * x;
        }
    }

    //Partial 2
    public partial class Matematik
    {
        public int kup_al(int x)
        {
            return x * x * x;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Matematik _m = new Matematik();
            Console.WriteLine(_m.kare_al(5));
            Console.WriteLine(_m.kup_al(7));
            Console.ReadLine();
        }
    }
}
```

Indexers (İndeksleyiciler)

- Bir indeksleyici, bir sınıfı dizi şeklinde kullanabilmek ve bu sınıftan türetilen nesneleri dizinleyebilmek amacıyla kullanılır. Başka bir deyişle bir indeksleyici, nesnelere dizi gibi davranılabilmesini sağlar.
- Indeksleyiciler tanımlanışları itibariyle, özelliklere (properties) çok benzerler. Ancak aralarında temel farklılıklarda vardır.

```
public string this[int Indeks]
{
    get
    {

    }
    set
    {

    }
}//this
```

This sınıf ismine yönlendirmedir. This indeksleyicinin tanımlandığı sınıfı belirtir.

İndeksleyici Kuralları

- Bir indeksleyici mutlaka bir geri dönüş tipine sahip olmalıdır. Yani bir indeksleyiciyi void olarak tanımlayamayız.
- Bir indeksleyiciyi static olarakta tanımlayamayız.
- Bir indeksleyici en az bir parametre almalıdır. Bununla birlikte, bir indeksleyici birden fazla ve çeşitte parametrede alabilmektedir.
- İndeksleyicileri aşırı yükleyebiliriz (Overloading). (Overloading konusu önümüzdeki haftalarda) Ancak bir indeksleyiciyi aşırı yüklediğimizde, bu indeksleyicileri birbirlerinden ayırırken ele aldığımız imzalar sadece parametreler ile belirlenir. İndeksleyicinin geri dönüş değeri bu imzada ele alınmaz.
- İndeksleyici parametrelerine , normal değişkenlermiş gibi davranamayız. Bu nedenle bu parametreleri ref ve out anahtar sözcükleri ile yönlendiremeyiz.
- Bir indeksleyici her zaman this anahtar sözcüğü ile tanımlamalıyız. Nitekim this anahtar sözcüğü , indeksleyicinin kullanıldığı sınıf nesnelerini temsil etmektedir. Böylece sınıfın kendisi bir dizi olarak kullanılabilir.

Indexers

```
public class Uye
{
    private string[] Uyeler = { "Metin", "Ali", "Feyyaz" };
    //Indeksleyici
    public string this[int Indeks]
    {
        get
        {
            if (Indeks < 0 || Indeks >= Uyeler.Length)
            {
                return null;
            }
            else
            {
                return Uyeler[Indeks];
            }
        }
        set
        {
            if (!(Indeks < 0 || Indeks >= Uyeler.Length))
                Uyeler[Indeks] = value;
        }
    }//this
    //Property
    public int Uzunluk
    {
        get { return Uyeler.Length; }
    }//Uzunluk
}//Uye
```

İndeks değeri kontrol ediliyor ve seçilen indeks değerindeki değer döndürülüyor. Eğer indeks dizi dışında ise null döndürülüyor.

İndeks değeri uygun değer ise dizi güncelleniyor.

Sadece okunabilir (readonly) bir property

Indexers

```
public class Uye
{
    private string[] Uyeler = { "Metin", "Ali", "Feyyaz" };
    //Indeksleyici
    public string this[int Indeks]
    {
        get
        {
            if (Indeks < 0 || Indeks >= Uyeler.Length)
            {
                return null;
            }
            else
            {
                return Uyeler[Indeks];
            }
        }
        set
        {
            if (!(Indeks < 0 || Indeks >= Uyeler.Length))
                Uyeler[Indeks] = value;
        }
    }//this
    //Property
    public int Uzunluk
    {
        get { return Uyeler.Length; }
    }//Uzunluk
}//Uye
```

```
class Program
{
    static void Main(string[] args)
    {
        Uye _u = new Uye();
        Console.WriteLine(_u[0]);
        _u[0] = "Mehmet";
        Console.WriteLine(_u[0]);
        Console.WriteLine(_u.Uzunluk);
        Console.ReadLine();
    }
}
```

Indexers - Overloading

- Bir string verilsin ve string dizi içinde arasın, eğer bulunursa indeks değeri döndürülsün. Aynı isme sahip fakat farklı parametrelere sahip(overloading) yeni bir indexers tanımlanabilir.

```
public class Uye
{
    private string[] Uyeler = { "Metin", "Ali", "Feyyaz" };
    //Indeksleyici 1
    public string this[int Indeks]...//this

    //Indeksleyici 2. overloading işlemi, ileride daha ayrıntılı anlatılacak.
    public int this[string aranan]
    {
        get
        {
            for (int i = 0; i < Uyeler.Length; i++)
            {
                if (aranan == Uyeler[i])
                    return i;
            }

            return -1; //Eğer hiç bir sonuç döndürmezse döndürülür.
        }
    }//this
    //Property
    public int Uzunluk...//Uzunluk
}//Uye
```

İlk yazdığımız indexer.
- özelliğine tıkladık.
+ ile tekrar kodu görebiliriz.

```
class Program
{
    static void Main(string[] args)
    {
        Uye _u = new Uye();
        Console.WriteLine(_u["Metin"]);

        Console.ReadLine();
    }
}
```

Kaynaklar

- https://bidb.itu.edu.tr/seyirdefteri/blog/2013/09/06/c-'ta-static-kavram%C4%B1
- http://www.buraksenyurt.com/post/C-Temelleri-Static-Olmak-bsenyurt-com-dan.aspx
- http://bidb.itu.edu.tr/seyirdefteri/blog/2013/09/08/isim-uzaylar%C4%B1-(namespaces)
- http://www.buraksenyurt.com/post/Indeksleyiciler-(Indexers)-bsenyurt-com-dan.aspx